

StockKeep

Shop Inventory & Stock Tracking System

Technical Debt Register & Repayment Plan

Technical_Debt_Plan.pdf — Standalone Document

Student Name:	[STUDENT NAME — TO BE PROVIDED]
Student ID:	[STUDENT ID — TO BE PROVIDED]
Project Title:	StockKeep — Shop Inventory & Stock Tracking System
Institution:	[INSTITUTION NAME — TO BE PROVIDED]
Department:	[DEPARTMENT NAME — TO BE PROVIDED]
Module / Course:	[MODULE / COURSE CODE — TO BE PROVIDED]
Supervisor:	[SUPERVISOR NAME — TO BE PROVIDED]
Date:	13 August 2026

Every debt item below was identified either by direct source-code inspection or by live testing evidence in *Testing_Report.pdf* (defects DEF-01 to DEF-05), and is cross-referenced to the requirement it violates in *SRS.pdf*. None is speculative.

Contents

1	Introduction	2
2	Technical Debt Register	3
3	Technical Debt Repayment Roadmap	11
3.1	Immediate	11
3.2	Version 1.1	11
3.3	Version 2.0	11
3.4	Long-Term	12
4	Alignment with Future Evolution	13

1. Introduction

StockKeep’s git history shows a single initial commit (“AI-Powered StockKeep Tracker with Analytics Dashboard”) followed immediately by four rapid hardening commits addressing Vercel deployment and AI-route reliability. This pattern — a full-featured system built in a short, fixed window and then patched reactively — is the direct cause of most debt items below. This document records that debt honestly rather than hiding it, and gives a prioritised, realistic plan to repay it.

2. Technical Debt Register

Debt ID	Technical Debt	Cause	Impact	Priority	Classification
TD-01	Login checks a hard-coded/env passcode (APP_PASSCODE, default 1234), never the Settings.passcode value the Settings screen lets a user edit. The two are completely disconnected.	Architecture shortcut — authentication was wired directly to an env constant early and the later Settings screen added a passcode field to the database without updating the login route to read it.	Store staff who change the passcode in Settings believe they have changed it, but the old passcode still works and the new one does nothing. Misleading, and a real security expectation gap.	Critical	Critical / Immediate
TD-02	POST /api/seed?force=true is reachable without authentication and deletes all Item, Supplier, Sale, and StockMovement records before reseeding. Confirmed exploitable live (Testing_Report.pdf, DEF-02).	Convenience shortcut to guarantee a populated demo on first Vercel request, whitelisted broadly in middleware.ts rather than narrowly.	Any unauthenticated actor who discovers the endpoint can wipe production data.	Critical	Critical / Immediate

Debt ID	Technical Debt	Cause	Impact	Priority	Classification
TD-03	No rate limiting, delay, or lockout on <code>/api/auth/login</code> . Confirmed live — 5 rapid wrong-passcode attempts all returned instantly (DEF-03).	Authentication simplification — a single shared 4-digit passcode was treated as “good enough” for the project’s time budget, without also budgeting for brute-force protection.	An attacker can brute-force a 4-digit numeric passcode in well under a minute at typical network speeds.	High	Critical / Immediate
TD-04	No CSRF token on state-changing routes; protection relies solely on the <code>SameSite=Lax</code> cookie attribute.	Authentication simplification, same root cause as TD-03.	Reduced but non-zero cross-site request forgery exposure on POST/PUT/DELETE routes.	Medium	Scheduled for Future Resolution
TD-05	No security response headers configured (<code>next.config.ts</code> has no <code>headers()</code> — no CSP, HSTS, X-Frame-Options). Confirmed live (DEF, Test-ing_Report TC-API-26).	Architecture shortcut — the default Next.js scaffold was never hardened post-scaffold.	Weaker defence-in-depth against clickjacking/XSS/protocol-downgrade in a production deployment.	Medium	Scheduled for Future Resolution

Debt ID	Technical Debt	Cause	Impact	Priority	Classification
TD-06	The Resend email API key is stored in plaintext in the Settings database table and returned verbatim by GET /api/settings to any authenticated caller (confirmed live, DEF-05), instead of being held as a server-side environment variable.	Dependency/configuration choice — third-party secret was modelled as ordinary per-store configuration data rather than a protected secret.	Any authenticated session (i.e., anyone with the shared passcode) can read the shop's real Resend API key.	High	Critical / Immediate
TD-07	Zero automated tests (no unit, integration, or E2E suite; no test script in package.json).	Limited automated testing — deliberately deferred under the project's time constraint in favour of shipping features.	No regression safety net; every change risks silently breaking existing behaviour (as DEF-01 in Testing_Report.pdf demonstrates was already possible).	High	Scheduled for Future Resolution

Debt ID	Technical Debt	Cause	Impact	Priority	Classification
TD-08	SQLite (prisma/dev.db, a local file) is used as the database provider while the stated deployment target is Vercel serverless, which has an ephemeral, largely read-only filesystem outside /tmp. The auto-seed-on-empty-database logic (ensureSampleDataSeeded) strongly suggests the team is already compensating for data not persisting reliably between deployments/cold starts.	Database/infrastructure decision made for local-development simplicity, not revisited for the production target.	Data written in production may not survive a redeploy or, depending on hosting configuration, may not be shared across serverless instances at all — a fundamental scalability/persistence limitation, not a bug that can be “fixed” without a database migration.	Critical	Critical / Immediate

Debt ID	Technical Debt	Cause	Impact	Priority	Classification
TD-09	The frontend is a single 1,921-line client component (<code>src/app/page.tsx</code>) handling nine views, all modals, and all state, with no component decomposition and no shared <code>src/components/</code> directory.	Time-constrained delivery — building one file was faster initially than setting up a component architecture.	Harder to test, harder to onboard a second developer to, higher risk of unrelated features interfering with each other as the file grows further.	Medium	Scheduled for Future Resolution
TD-10	<code>items GET</code> fetches the entire table into memory and filters/search in JavaScript rather than pushing filters into the Prisma query; <code>sales GET</code> hardcodes <code>take: 100</code> with no pagination controls exposed to the client.	Incomplete refactoring — filtering was implemented against an in-memory array early on (when the seed dataset was small) and never migrated to query-level filtering as the schema and dataset grew.	Will not scale past a small inventory; no way for a user to page through more than 100 recent sales.	Medium	Scheduled for Future Resolution

Debt ID	Technical Debt	Cause	Impact	Priority	Classification
TD-11	<code>insightsCache</code> (5-minute AI insights cache) is a plain module-level JavaScript variable.	Temporary solution — an in-memory cache is the simplest possible implementation and works fine in a single long-lived process.	On Vercel serverless, each function instance has its own memory and can cold-start at any time, so the “5-minute cache” does not behave consistently in production — different requests may hit different instances with different (or no) cached state.	Low	Acceptable Temporarily
TD-12	Several <code>catch</code> blocks silently swallow errors (<code>.catch(() => {})</code>) on the fire-and-forget seed call in <code>login/page.tsx</code> without also <code>.catch(console.error)</code> on the low-stock email send) — failures are logged server-side only and never surfaced to the user.	Error handling was scoped down to “don’t let a side-effect break the main request” without also budgeting for user-facing visibility of that failure.	A shop owner relying on low-stock email alerts (FR-14) may never realise the emails have silently stopped sending (e.g., after DEF-04’s Gemini/API-key style misconfiguration, or an invalid Resend key).	Medium	Scheduled for Future Resolution

Debt ID	Technical Debt	Cause	Impact	Priority	Classification
TD-13	The Gemini model name (<code>gemini-1.5-flash</code>) is hardcoded in <code>src/lib/geminiphs</code> and, per live testing (DEF-04), currently returns 404 Not Found from the real API for this project's key/API version — every AI feature is silently running on its fallback path only.	Dependency choice not revisited — the model name was chosen once and never re-validated against the provider's current supported-model list.	The “AI-powered” features advertised in the project title are not currently producing AI-generated output in practice, only deterministic/rule-based fallbacks, even though the code path for real AI output is otherwise correctly implemented.	High	Critical / Immediate
TD-14	<code>README.md</code> is the unmodified default <code>create-next-app</code> boilerplate; no project-specific setup, environment-variable, or architecture documentation exists outside this documentation package.	Documentation gap — normal for a time-boxed build, but never revisited afterward.	A new developer (or the marker) has no in-repo entry point beyond source reading; this documentation package is currently the <i>only</i> place these facts are recorded.	Low	Scheduled for Future Resolution

Debt ID	Technical Debt	Cause	Impact	Priority	Classification
TD-15	Several route handlers type caught errors as <code>catch (error: any)</code> , opting out of the strict TypeScript checking enabled elsewhere (<code>tsconfig.json</code> , <code>"strict": true</code>).	Convenience shortcut around TypeScript's stricter error-typing ergonomics.	Slightly weakens the type-safety guarantees the rest of the codebase relies on; low real-world impact but inconsistent with the project's own stated strictness.	Low	Acceptable Temporarily

3. Technical Debt Repayment Roadmap

3.1 Immediate

Fix before any real shop's data is trusted to a live deployment:

- **TD-01** — Make login check `Settings.passcode` (hashed) instead of `/` in addition to `APP_PASSCODE`, and hash all passcodes at rest (bcrypt/argon2) rather than plaintext comparison.
- **TD-02** — Remove `/api/seed` from the unauthenticated middleware whitelist, or require a signed one-time admin token specifically for `force=true`.
- **TD-03** — Add basic login attempt throttling (e.g., a fixed lockout window after 5 failed attempts per IP/session).
- **TD-06** — Move the Resend API key out of the `Settings` table into a server-only environment variable; stop returning it from `GET /api/settings`.
- **TD-08** — Decide and document the real production persistence strategy (e.g., migrate to Vercel Postgres / Neon / Turso, or explicitly scope the deployment to a single always-on server rather than serverless) before treating any deployed data as durable.
- **TD-13** — Confirm which Gemini model the configured API key actually has access to and update `src/lib/gemini.ts`; add a one-line startup log so a misconfigured model is visible immediately rather than only inferred from response latency.

3.2 Version 1.1

Next planned iteration, once Immediate items are resolved:

- **TD-04** — Add CSRF tokens (or migrate fully to a same-site-only architecture with `SameSite=Strict` where UX allows) on all state-changing routes.
- **TD-05** — Add security headers via `next.config.ts headers()` (CSP, X-Frame-Options, Referrer-Policy, HSTS on the production domain).
- **TD-07** — Introduce an automated test suite (Vitest or Jest for unit tests on `src/lib/validations.ts` and `src/lib/seed.ts`; Playwright for the core login → add item → adjust stock → sale flow), starting from the 26 manual test cases already documented in `Testing_Report.pdf` so existing coverage is not lost, only automated.
- **TD-12** — Surface currently-silent failures (seed trigger, low-stock email) to the UI or an admin-visible log, rather than only the server console.

3.3 Version 2.0

Structural work once the product direction is validated:

- **TD-09** — Decompose `src/app/page.tsx` into a proper `src/components/` tree (one component per view/modal), enabling code-splitting and independent testing.

- **TD-10** — Move filtering/search/pagination for `items` and `sales` into the Prisma query layer (`where`, `skip/take`) and expose pagination controls in the UI.
- **TD-11** — Replace the in-memory `insightsCache` with a shared cache appropriate to the serverless target (e.g., Vercel KV/Redis) if the 5-minute caching behaviour is still desired at scale.

3.4 Long-Term

Dependent on real usage/scale, revisited after Version 2.0 ships:

- Introduce genuine multi-user accounts and role-based access control if the single-shared-passcode model no longer fits the shop's needs (this is a product decision, not only a technical one).
- Reassess SQLite-vs-managed-database (TD-08) again in light of actual measured usage once on a durable backend, to confirm the chosen database still fits the read/write pattern.
- **TD-14, TD-15** — Backfill project documentation into the repository itself (a real `README.md`, an `ARCHITECTURE.md`) and clean up remaining `catch (error: any)` typing as the codebase stabilises.

4. Alignment with Future Evolution

The roadmap above is sequenced so that every “Immediate” item removes a risk that would otherwise undermine trust in the platform (data loss, credential exposure, unauthenticated destructive actions) before any feature work continues. Version 1.1 turns the ad-hoc manual testing and security posture established during this documentation pass into durable, automated safeguards. Version 2.0 pays down the structural (frontend architecture, query-layer) debt that would otherwise make every future feature progressively more expensive to add. This ordering is deliberate: it is cheaper to restructure `page.tsx` (TD-09) once the underlying auth/data risks (TD-01, TD-02, TD-08) are no longer in flux, since those fixes will themselves touch several of the same routes.